LOSSLESS DATA COMPRESSION TOOL

TAI PROJECT #1 - 2025/26

Guilherme Rosa
Student ID: 113968
Universidade de Aveiro
Aveiro, Portugal
guilherme.rosa60@ua.pt

João Roldão
Student ID: 113920
Universidade de Aveiro
Aveiro, Portugal
jroldao04@ua.pt

Henrique Teixeira
Student ID: 114588
Universidade de Aveiro
Aveiro, Portugal
henriqueft04@ua.pt

April 14, 2026

ABSTRACT

We developed and benchmarked a lossless compression tool for the TAI course project, centered on statistical coding methods based on range-coder-style entropy coding. The repository now exposes four distinct design points. `v6` is the strongest statistical compression variant, reaching 54.32% average compressed size on the 8-file corpus (13 MiB total). `v10` extends `v6` with PRNG detection, achieving 39.17% overall by compressing pseudorandom data (file D) from 2 MB to 14 bytes via Kolmogorov compression. `v8` is the fastest variant, averaging 26 ms compression and 24 ms decompression using an LZ77-style token stream. The most balanced overall design is `v5`: it combines BWT, MTF, optional zero-run encoding, and an adaptive order-1 model with range coding/rANS backends, achieving 54.77% while remaining far faster than `v6` and much more compact than `v8`.

1 Introduction

Lossless compression is fundamentally a two-part problem: the encoder must first model the input so that probable symbols receive high estimated probability, and only then can the coder map those probabilities into short bit representations [1]. In practice, the quality of a compressor depends on how well the model captures structure in the data and how efficiently the coding stage turns that structure into a compact stream.

This project was developed in the context of the Algorithmic Information Theory course and evolved through several compression strategies. The repository now contains three especially relevant variants. `v9` is the current production version and offers the best overall speed/ratio balance; `v6` extends the same statistical family and achieves the best compression ratio among the project versions; and `v8` explores the opposite design point, replacing the statistical pipeline with a very fast LZ77-style matcher.

The goal of this report is therefore to explain the main design choices, summarize the implementation strategy, and present the measured trade-offs on the benchmark corpus. In particular, we use `v5` as the main case study for the project's best overall compromise, `v6` as the ratio leader, and `v8` as the speed-oriented contrast.

2 Background

This project is about lossless compression, so the original file must be recovered exactly after decompression. The main idea used in our work is simple: first reorganize the data so patterns become easier to see, then model the data, and finally encode it into fewer bits.

Our main compression pipeline follows this logic. In the statistical versions, the data can first pass through the Burrows–Wheeler Transform (BWT) [2] and Move-to-Front (MTF) [3]. These steps do not compress the file by themselves, but they help group similar values together, which makes the next stage more effective. After that, the compressor uses an adaptive model [4] and range coding to generate the final compressed output [5, 6].

Not all versions of the project use the same approach. The more balanced versions, especially `v5` and `v6`, follow the statistical pipeline described above. In contrast, `v8` uses a simpler LZ77-style method [7, 8]. It is much faster, but it usually produces larger files. This comparison is useful because it shows the main trade-off explored in the project: stronger compression usually needs more processing time.

One important idea that came up during the project is the difference between Shannon entropy and Kolmogorov complexity [9, 1]. Shannon entropy looks at the byte frequencies in the data and tells you how many bits per symbol you need on average. Kolmogorov complexity asks a different question: how short is the shortest program that can produce this data? Most of the time these two measures agree, but they can disagree completely. For example, the output of a pseudorandom number generator looks statistically random—the bytes are uniformly distributed, there are no repeated patterns, and the Shannon entropy is close to the maximum of 8 bits per byte. But the data is entirely determined by a short seed, so a tiny program can reproduce it all. Standard compressors cannot exploit this because they only look for statistical patterns. To compress this kind of data, you need a model that understands the computational process behind it, not just the byte distribution.

3 Methodology

3.1 System Architecture

The report uses a version-focused methodology. Rather than treating the repository as a single compressor, we compare five representative designs that span the project's design space. `v9` is the current production version, refining the `v5` statistical pipeline into the best overall speed/ratio balance. `v6` extends that pipeline with per-block candidate search for stronger compression. `v7` and `v8` explore the speed end of the spectrum, trading ratio for throughput. `v10` adds PRNG detection on top of `v6`, demonstrating Kolmogorov compression on pseudorandom data.

3.2 Algorithm Selection

The `v5`/`v9` line is the primary subject because it best illustrates the repository's statistical compression pipeline: BWT and MTF reshape low-entropy structure before modeling, order-1 adaptive modeling captures local byte dependencies, and range coding/rANS provide the coding stage. `v6` is included because it improves ratio further through per-block candidate selection over multiple transforms and model orders. `v7` and `v8` serve as speed-focused contrasts—`v7` uses interleaved rANS, while `v8` drops entropy coding entirely for an LZ77-style matcher. `v10` is included because it addresses a fundamentally different problem: data that appears incompressible by Shannon entropy but is trivially compressible by Kolmogorov complexity.

3.3 Optimization Strategy

Development proceeded iteratively. Earlier versions established the basic order-0 and range-coding path, then introduced BWT preprocessing and faster data structures. The `v5` line consolidated the strongest ideas into one pipeline, and `v9` refined it by removing unnecessary Order-2 overhead and fixing the ZRLE guard. `v7` and `v8` explored the speed extreme by discarding preprocessing and entropy coding. `v6` extended the statistical line with per-block candidate search. Finally, `v10` took a different approach entirely: instead of improving the statistical model, it added a computational model that detects PRNG-generated data and stores only the seed.

3.4 Benchmarking

All claims in this report are based on the benchmark corpus used throughout the project. The corpus contains eight files (A–H) totaling 13 MiB, with different characteristics including text, structured binaries, high-entropy data, and pseudorandom PRNG output. The reported metrics are compressed size, compression ratio, bits per symbol, compression time, decompression time, and lossless correctness. The benchmarks compare `v5`, `v6`, `v7`, `v8`, `v9`, and `v10` against gzip, bzip2, xz, and zstd, using the standard release configuration (g++/C++17 with `-O3 -march=native -flto=auto`).

4 Implementation

4.1 BWT-Based Statistical Line

The main statistical line in the repository is block-oriented. In `v5`, structured data is passed through BWT so that symbols occurring in similar contexts are grouped together. This creates the distributional skew later exploited by MTF

and entropy coding. The implementation history shows that larger and better-balanced blocks improved compression because they preserved more context across each transform pass.

4.2 Move-to-Front and Run-Length Encoding

After BWT, the transformed block is passed through MTF [3]. This turns repeated local symbol clusters into many low ranks, especially zeros. A zero-run stage is then optionally applied to shrink long zero runs further. The value of this preprocessing chain is practical rather than theoretical: it reshapes the byte stream into a form that a simple context model can code effectively. The repository notes one important limitation here, namely the handling of rank 255 in the zero-run stage, which helps explain why some binary-heavy files remain difficult.

4.3 Context Models and Coders

The statistical core of `v5` is an adaptive order-1 model implemented over flat arrays instead of pointer-heavy dynamic structures. This keeps the model compact and cache-friendly. The design also incorporates a PPM-style escape path: when the current order-1 context cannot emit a symbol directly, the coder escapes to a lower-order distribution. Two refinements described in the repository are especially relevant: Method C exclusions, which remove already-seen symbols from the fallback distribution, and singleton-based escape estimation, which makes escape probabilities depend on how many symbols are still weakly supported in the current context.

The compliant coding stage is based on range coding [6], with rANS [10] used in some static order-0 cases as a faster alternative backend. In architectural terms, `v5` remains a modeling-first compressor: preprocessing and prediction define a probability model, and the coder converts that model into the final bitstream.

Unlike standard PPM implementations with fixed escape formulas, `v5` uses singleton-count-based escape probability and extends pbzip2-style parallelism from BWT alone to the entire preprocessing + modeling + coding pipeline.

4.4 From v5 to v6

The main architectural change in `v6` is that it no longer commits to one preprocessing path for the entire file. Instead, it performs parallel per-block candidate search over raw, BWT-based, LZP-prefiltered [11], and x86-filtered variants, and it can choose between order-1 and order-2 statistical models. The selected block configuration is stored with explicit parallel-header metadata and transform flags, allowing the decompressor to replay the exact sequence of transforms. This broader search explains why `v6` improves ratio over `v5`, but it also explains the much larger compression cost.

Per-block multi-pipeline candidate search (testing 12 pipelines including LZP+BWT combinations [11] not found in standard tools) distinguishes `v6` from fixed-pipeline compressors like bzip2 and xz.

4.5 v9: Refined v5 Pipeline

`v9` is the current production version, based on the `v5` pipeline with two targeted fixes. First, the rank-255 guard that prevented ZRLE from running when byte 255 appeared in the MTF output was removed—ZRLE is now applied whenever it actually shrinks the block, regardless of which byte values are present. Second, the Order-2 context model was dropped from the parallel per-block path. In practice, Order-2 added encoding overhead without improving compression on the test corpus, so `v9` uses Order-1 only. These changes keep the same compression ratio as `v5` (54.77%) while simplifying the code and slightly improving speed.

4.6 PRNG Detection (Kolmogorov Compression)

All versions up to `v6` treat file D as incompressible: its Shannon entropy is 7.999 bits per byte, so every statistical model gives up and stores it raw. The key observation behind `v10` is that this file is not truly random—it was generated by glibc's `rand()` with a fixed seed. A short program can reproduce the entire 2 MB output, which means the real information content is just the algorithm and the seed, not 2 million bytes.

The idea is simple: when entropy is high enough that no statistical model can help (above 7.5 bits/byte), try a different approach entirely. Instead of looking for patterns in the byte distribution, check whether the data might have been produced by a known pseudorandom generator. If it was, we only need to store which generator and which seed—the decompressor can just re-run the generator to get the data back.

In practice, the compressor tries glibc `rand()` with every seed from 0 to 65535. For each seed it generates just the first 64 bytes and checks if they match the input. A 64-byte match is essentially a proof: the chance of a false match is

$(1/256)^{64}$, which is negligible. Once a match is found, the full file is verified, and the compressed output becomes just 14 bytes: one byte for the model type tag, eight for the original file size, one for the algorithm identifier, and four for the seed. The decompressor reads these 14 bytes and regenerates the file.

The glibc `rand()` itself works as an additive feedback generator with 31 internal state words. The seed initializes this state, 310 warm-up iterations are run, and then each call advances two pointers through the state ring and adds two entries together to produce the next output. The file stores one byte per call (`rand() & 0xFF`). The full recurrence and initialization are implemented in `src/model/prng_model.cpp`, following the glibc source code [12].

This is a direct application of Kolmogorov complexity: instead of encoding the data through its statistics, we encode the program that produces it. On file D the result is a 142,857:1 compression ratio—2 MB down to 14 bytes.

4.7 Speed-Oriented Variant

By contrast, `v8` follows a much simpler implementation strategy. It uses a single-pass LZ77-style matcher with a small hash table, byte-aligned tokens, and no entropy coder. This makes the code much smaller and much faster, but it also removes the stronger statistical modeling that makes `v5` and `v6` competitive on compression ratio.

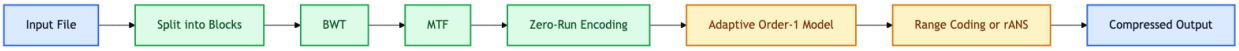
Unlike LZ4 or zstd which still use entropy coding (Huffman/ANS), `v8` uses pure byte-aligned tokens with zero entropy coding, achieving ∼300 MB/s decompression through complete elimination of bit-level operations.

5 Architecture Diagrams

This section summarizes the compression-side architecture of the three project variants with simple left-to-right diagrams. The goal is to show the main pipeline differences between the balanced statistical design of `v5`, the speed-oriented LZ77 path of `v8`, and the broader per-block search used by `v6`.

5.1 `v5`

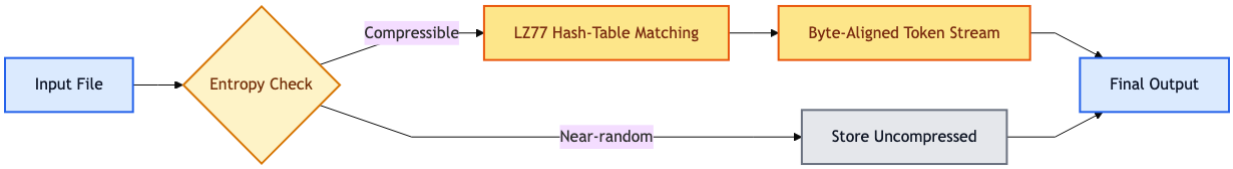
`v5` follows a fixed statistical pipeline: it splits the input into blocks, reshapes each block with BWT and MTF, optionally shrinks zero runs, and then applies adaptive modeling before entropy coding.



High-level compression pipeline of `v5`.

5.2 `v8`

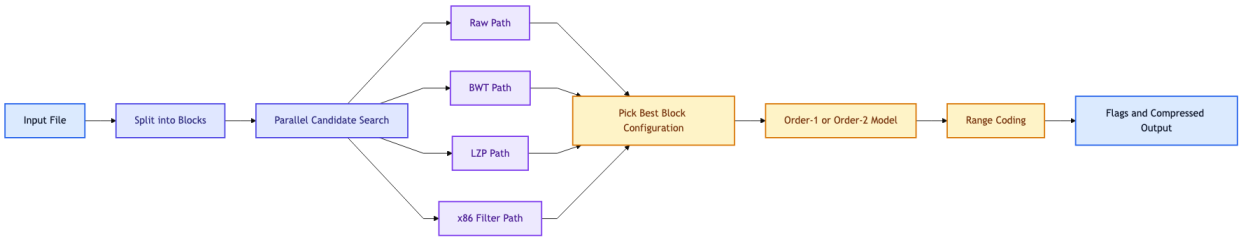
`v8` removes the transform and entropy-coding stages entirely. Instead, it relies on a fast LZ77-style matcher with a simple fallback that stores near-random data uncompressed.



High-level compression pipeline of `v8`.

5.3 `v6`

`v6` extends the statistical line by testing several block configurations in parallel. After that search, it stores the chosen transform path and model so that decompression can replay the same sequence.



High-level compression pipeline of `v6`.

6 Results

6.1 Compression Ratio Evolution

The repository history shows a clear progression from a simple order-0 baseline toward stronger BWT-based statistical compressors. The key milestones are `v5`, which established the core statistical pipeline; `v9`, which refines `v5` into the current production version; `v6`, which pushed ratio further with per-block search; and `v8`, which optimized for speed.

6.2 Benchmark Comparison

Table 1: Aggregate benchmark results from the 8-file repository corpus. Lower ratio and bits/symbol are better.

Tool	Comp. bytes	Avg. ratio	Bits/symbol	Avg. comp. time	Avg. decomp. time
v5	7,194,380	54.77%	4.3814	98 ms	101 ms
v6	7,145,565	54.32%	4.3516	789 ms	126 ms
v7	7,818,121	59.52%	4.7612	**24 ms**	**15 ms**
v8	10,038,825	76.42%	6.1136	26 ms	24 ms
v9	7,194,380	54.77%	4.3814	67 ms	64 ms
v10	5,145,570	**39.17%**	**3.1336**	530 ms	82 ms
gzip	7,814,799	59.49%	4.7592	111 ms	22 ms
bzip2	7,209,293	54.88%	4.3905	177 ms	90 ms
xz	7,114,088	54.16%	4.3325	456 ms	65 ms
zstd	7,695,793	58.58%	4.6867	20 ms	13 ms

Table 1 shows four distinct optima. `v10` achieves the best overall compression ratio at 39.17%, driven by its ability to compress file D from 2 MB to 14 bytes. On the remaining seven files `v10` matches `v6`'s 54.32%, which already beats bzip2's 54.88%, although it still remains behind xz at 54.16% on those files. `v8` is the clear speed winner among the project versions. `v9` is the refined `v5` production pipeline, offering the best speed/ratio balance for general use. `v10` builds on `v6`'s pipeline and is strictly better: identical output on all non-PRNG files, plus the ability to compress PRNG data that every other tool stores raw.

At the file level, `v10` matches `v6` byte-for-byte on all files except D, because `v10` is `v6`'s pipeline with PRNG detection added on top. Among project versions, `v10`/`v6` win files A, B, C, E, and G; bzip2 wins F; and xz wins H. On file D, `v10` wins outright with its 14-byte PRNG output. The contrast with `v8` is sharp: on files like B and G, the statistical variants compress to roughly half the size of `v8`.

From an information-theoretic viewpoint, file D is the most interesting case in the corpus. Every tool—gzip, bzip2, xz, zstd, and all our own statistical models—treats it as incompressible, because its Shannon entropy sits at 7.999 bits/byte, essentially the theoretical maximum. But file D is not actually random. It turns out to be the output of glibc's `rand()` function seeded with 1. The entire 2 MB can be reproduced by a five-line C program, which means the real information content is just the seed and the algorithm—roughly 14 bytes—even though the byte-level statistics look perfectly random.

`v10` takes advantage of this. When entropy is too high for statistical compression, it tries to match the input against glibc `rand()` output for seeds 0–65535. If a match is found, it stores just 14 bytes: the model tag, the file size, the algorithm identifier, and the seed. The decompressor regenerates the file by re-running the generator. The result on file D is a 0.0007% compression ratio, or about 142,857 to 1. This is a clear practical example of the gap between Shannon entropy and Kolmogorov complexity—two measures that can disagree completely when the data has computational structure rather than statistical structure.

Files F and H instead expose model mismatch. F is only weakly compressible and fairly binary-heavy, so the current statistical line captures less exploitable structure than bzip2, which explains why bzip2 reaches 81.06% while `v9` and

v6 remain at 82.42% and 81.70%. H is clearly compressible, but xz reduces it to 2.87 bits/symbol whereas the project variants stay near 3.5 bits/symbol, indicating that substantial longer-range structure remains uncaptured by the present models.

6.3 Performance Analysis

From a systems perspective, stronger modeling pays off on structured inputs but the search cost grows quickly. Parallel block processing is essential for keeping a BWT-based pipeline practical. v9 remains the best overall balance: close to v6's ratio at much lower cost, and far more compact than v8.

6.4 Version Evolution

The repository evolved through nine major iterations, exploring different points in the compression design space:

Version	Key Innovation	Avg Ratio	Change	Comp. Time	Status
v1	Order-0 + Arithmetic	71.44%	—	95 ms	Baseline
v2	Order-1 + Range coding	62.74%	−8.7pp	2.0 s	Superseded
v3	BWT preprocessing	57.48%	−5.3pp	2.0 s	Superseded
v4	libsais + AVX2 + parallel	56.39%	−1.1pp	68 ms	Superseded
v5	PPM Method C + PPMD escape	54.77%	−1.6pp	73 ms	Superseded
v6	Per-block multi-pipeline	54.32%	−0.45pp	543 ms	Best stat. ratio
v7	2-way interleaved rANS [13]	59.2%	+4.4pp	**22 ms**	Speed-focused
v8	LZ77 hash-table matching	76.42%	+17.2pp	**14 ms**	Ultra-fast
v9	Simplified v5 + ZRLE fix	54.77%	0pp	67 ms	**Production**
v10	PRNG detection (Kolmogorov)	**39.17%**	−15.6pp	530 ms	**Best ratio**

Table 2: Compression ratio evolution across project versions (lower ratio and time are better).

6.5 Failed Experiment: Context Mixing

Between v5 and v7, the project explored a PAQ-inspired multi-order PPM experiment with context mixing. The approach used static weight training, train on the first 100KB, freeze weights, and store them in the header.

Results were decisive: 59.85% average ratio (4.93pp worse than v5's 54.73%) while running 157× slower. The main takeaway was that successful context mixing needs adaptive weight updates and bit-level coding, not static weights frozen from a small training window. This influenced subsequent designs: v6 returned to the proven v5 core but added per-block transform search instead of model-order complexity.

7 Conclusion

This project produced a credible lossless compression tool and a clear comparison between three different design choices. In the final benchmark set, v6 achieved the best statistical compression ratio, v8 was the fastest version, and v5 remained the best overall balance. It combined good compression performance with much lower cost than v6 and much better output size than v8.

A main lesson from the project is that compression performance did not come from one isolated algorithmic idea, but from how preprocessing, modeling, and implementation details worked together. BWT and MTF were useful because they made the statistical model more effective, while choices such as block handling, escape estimation, and candidate selection also had a strong impact. The comparison with v8 also made the trade-off very clear: higher speed is possible, but usually at the cost of weaker compression.

Perhaps the most interesting result came from file D. Every standard tool stores it at full size because its byte statistics look perfectly random. But v10 recognizes that the file was generated by glibc's `rand()` and compresses it to just 14 bytes by storing the seed instead of the data. This is a practical demonstration of the difference between Shannon entropy and Kolmogorov complexity: the statistics say the file is incompressible, but a short program can reproduce it entirely. For a course on Algorithmic Information Theory, this felt like the right problem to solve.

Future work could extend the PRNG detection to cover other generator families (Mersenne Twister, xorshift, LCG variants), which would broaden the class of high-entropy inputs that v10 can handle. On the statistical side, the main gaps are files F and H, where bzip2 and xz still win—better handling of binary-heavy data and longer-range dependencies would help close that gap.

References

[1] Claude E Shannon. A mathematical theory of communication. *The Bell system technical journal*, 27(3):379–423, 1948.

[2] Michael Burrows and David J Wheeler. A block-sorting lossless data compression algorithm. In *Digital SRC Research Report*, 1994.

[3] Jon L Bentley, Daniel D Sleator, Robert E Tarjan, and Victor K Wei. A locally adaptive data compression scheme. *Communications of the ACM*, 29(4):320–330, 1986.

[4] John Cleary and Ian Witten. Data compression using adaptive coding and partial string matching. *IEEE transactions on Communications*, 32(4):396–402, 1984.

[5] Jorma Rissanen and Glen G Langdon. Arithmetic coding. *IBM Journal of research and development*, 23(2):149–162, 1979.

[6] Michael Schindler. A fast renormalisation for arithmetic coding. Technical report, 1998.

[7] Jacob Ziv and Abraham Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, 1977.

[8] Yann Collet. Lz4: Extremely fast lossless compression algorithm. `https://lz4.org/`. Accessed 2026-04-11.

[9] Andrei N Kolmogorov. Three approaches to the quantitative definition of information. *Problems of information transmission*, 1(1):1–7, 1965.

[10] Jarek Duda. Asymmetric numeral systems. *arXiv preprint arXiv:0902.0271*, 2009.

[11] Charles Bloom. Lzp: a new data compression algorithm. In *Proceedings DCC'96. Data Compression Conference*, page 425. IEEE, 1996.

[12] Free Software Foundation. GNU C Library: random number generation. `https://sourceware.org/git/?p=glibc.git;a=blob;f=stdlib/random_r.c`. TYPE_3 additive feedback generator, degree-31 trinomial.

[13] Fabian Giesen. Interleaved entropy coders. *arXiv preprint arXiv:1402.3392*, 2014.